DSA MASTER SHEET

100 Java Coding Questions — Offline Edition

Standalone problem statements • Examples • Hints • Java methods

This version contains the questions themselves. It does not print the external website links. Each problem is rewritten as a self-contained practice prompt based on the topic in your original PDF, with an example test case and a Java solution or implementation pattern.

How to practise: Read the statement first, write your own solution, then check the hint and Java method. Examples are illustrative; platform-specific constraints and exact judge input/output formats may differ.

Note: The original sheet's Question 100 points to a general Codeforces problem list, not a single fixed problem. It is therefore marked as a choose-a-problem exercise.

EASY — QUESTIONS 1–25

1. Rotate Array by K

Problem statement: Given an integer array and a non-negative integer k, rotate the array to the right by k positions. The operation must be done in-place, using constant extra space.

Example test case: [1,2,3,4,5,6,7], k=3 → [5,6,7,1,2,3,4]

Hint / algorithm: Reverse the whole array, then the first k items, then the rest.

Java solution or implementation pattern:

```
static void rotate(int[] a,int k){int n=a.length;if(n==0)return;k%=n;rev(a,0,n-1);rev(a,0,k-1);rev(a,k,n-1);} static void rev(int[] a,int l,in
```

2. Move Zeroes

Problem statement: Given an integer array, move every zero to the end of the array while preserving the relative order of all non-zero elements. Modify the array in-place.

Example test case: [0,1,0,3,12] → [1,3,12,0,0]

Hint / algorithm: Write nonzero values forward, then fill remaining positions with zero.

Java solution or implementation pattern:

```
static void moveZeroes(int[] a){int w=0;for(int x:a)if(x!=0)a[w++]=x;while(w<a.length)a[w++]=0;}
```

3. Second Largest Element

Problem statement: Given an integer array, return its second-largest distinct value. If the array contains fewer than two distinct values, report that no second-largest value exists.

Example test case: [12,35,1,10,34,1] → 34

Hint / algorithm: Track largest and second-largest distinct values.

Java solution or implementation pattern:

```
static int secondLargest(int[] a){int max=Integer.MIN_VALUE,second=Integer.MIN_VALUE;for(int x:a){if(x>max){second=max;max=x;}else if(x<max&&x
```

4. Remove Duplicates from Sorted Array

Problem statement: Given a sorted integer array, remove duplicate values in-place so that each distinct value appears once in the beginning of the array. Return the number of distinct values.

Example test case: [1,1,2,2,3] → length 3, prefix [1,2,3]

Hint / algorithm: Two pointers; write each newly seen value at the next slot.

Java solution or implementation pattern:

```
static int removeDuplicates(int[] a){if(a.length==0)return 0;int w=1;for(int r=1;r<a.length;r++)if(a[r]!=a[w-1])a[w++]=a[r];return w;}
```

5. Missing Number

Problem statement: An array contains n distinct integers chosen from the range 0 through n, with exactly one number missing. Find and return the missing number.

Example test case: [3,0,1] → 2

Hint / algorithm: XOR all indices and values.

Java solution or implementation pattern:

```
static int missingNumber(int[] a){int x=a.length;for(int i=0;i<a.length;i++)x^=i^a[i];return x;}
```

6. Find Duplicate Number

Problem statement: An array of length n+1 contains integers from 1 through n. At least one value is repeated. Find a duplicate without modifying the array and using constant extra space.

Example test case: [1,3,4,2,2] → 2

Hint / algorithm: Floyd cycle detection on index links.

Java solution or implementation pattern:

```
static int findDuplicate(int[] a){int slow=a[0],fast=a[0];do{slow=a[slow];fast=a[a[fast]];}while(slow!=fast);slow=a[0];while(slow!=fast){slow=
```

7. Check Sorted and Rotated

Problem statement: Given an integer array, determine whether it could be a non-decreasing sorted array after being rotated any number of positions.

Example test case: [3,4,5,1,2] → true

Hint / algorithm: Count circular descents; at most one is allowed.

Java solution or implementation pattern:

```
static boolean sortedRotated(int[] a){int drops=0,n=a.length;for(int i=0;i<n;i++)if(a[i]>a[(i+1)%n])drops++;return drops<=1;}
```

8. Majority Element

Problem statement: Given an array in which a majority element is guaranteed to exist, return the element that occurs more than floor(n/2) times.

Example test case: [2,2,1,1,1,2,2] → 2

Hint / algorithm: Boyer–Moore voting.

Java solution or implementation pattern:

```
static int majorityElement(int[] a){int c=0,cand=0;for(int x:a){if(c==0)cand=x;c+=x==cand?1:-1;}return cand;}
```

9. Two Sum

Problem statement: Given an integer array and a target, return the indices of two different elements whose sum equals the target. Assume exactly one valid pair exists.

Example test case: [2,7,11,15], target=9 → [0,1]

Hint / algorithm: HashMap stores previously seen value-to-index mappings.

Java solution or implementation pattern:

```
static int[] twoSum(int[] a,int target){Map<Integer,Integer> m=new HashMap<>();for(int i=0;i<a.length;i++){int need=target-a[i];if(m.containsK
```

10. Longest Prefix Also Suffix (LPS)

Problem statement: Given a string s, find the length of the longest proper prefix of s that is also a suffix. The prefix must not be the entire string.

Example test case: "abab" → 2

Hint / algorithm: Build the KMP prefix-function array.

Java solution or implementation pattern:

```
static int lps(String s){int[] p=new int[s.length()];for(int i=1,j=0;i<s.length();){if(s.charAt(i)==s.charAt(j))p[i++]=++j;else if(j>0)j=p[j-1
```

11. Valid Palindrome

Problem statement: Given a string, determine whether it is a palindrome after ignoring spaces, punctuation, and letter case. Consider only letters and digits.

Example test case: "A man, a plan, a canal: Panama" → true

Hint / algorithm: Two pointers skip irrelevant characters.

Java solution or implementation pattern:

```
static boolean isPalindrome(String s){int l=0,r=s.length()-1;while(l<r){while(l<r&&!Character.isLetterOrDigit(s.charAt(l)))l++;while(l<r&&!Cha
```

12. Reverse Words in a String

Problem statement: Given a string containing words separated by spaces, return the words in reverse order with exactly one space between words and no leading or trailing spaces.

Example test case: " hello world " → "world hello"

Hint / algorithm: Trim and split on whitespace, then reverse words.

Java solution or implementation pattern:

```
static String reverseWords(String s){String[] w=s.trim().split("\\s+");Collections.reverse(Arrays.asList(w));return String.join(" ",w);}
```

13. Valid Anagram

Problem statement: Given two lowercase English strings, determine whether one is an anagram of the other; that is, whether both contain the same letters with the same frequencies.

Example test case: "anagram", "nagaram" → true

Hint / algorithm: Count characters; this version supports lowercase English letters.

Java solution or implementation pattern:

```
static boolean isAnagram(String a,String b){if(a.length()!=b.length())return false;int[] c=new int[26];for(char x:a.toCharArray())c[x-'a']++;f
```

14. Frequency of Array Elements

Problem statement: Given an integer array, count how many times each distinct value occurs. Return each value with its frequency.

Example test case: [1,2,2,3,1,1] → {1=3, 2=2, 3=1}

Hint / algorithm: Use a HashMap for arbitrary integer values.

Java solution or implementation pattern:

```
static Map<Integer,Integer> frequency(int[] a){Map<Integer,Integer> m=new LinkedHashMap<>();for(int x:a)m.put(x,m.getOrDefault(x,0)+1);return
```

15. First Non-Repeating Character

Problem statement: Given a string, return the index of its first character that occurs exactly once. If every character repeats, return -1.

Example test case: "leetcode" → 0

Hint / algorithm: Count then scan in original order.

Java solution or implementation pattern:

```
static int firstUnique(String s){int[] c=new int[Character.MAX_VALUE+1];for(char x:s.toCharArray())c[x]++;for(int i=0;i<s.length();i++)if(c[s
```

16. First and Last Occurrence

Problem statement: Given a sorted integer array and a target, return the first and last positions of the target. If the target is absent, return [-1, -1]. Your algorithm should run in O(log n) time.

Example test case: [5,7,7,8,8,10], target=8 → [3,4]

Hint / algorithm: Use lower-bound and upper-bound binary searches.

Java solution or implementation pattern:

```
static int[] range(int[] a,int t){return new int[]{bound(a,t,false),bound(a,t,true)};} static int bound(int[] a,int t,boolean upper){int l=0,
```

17. Search in Nearly Sorted Array

Problem statement: A nearly sorted array is one in which each element may be displaced from its sorted position by at most one index. Find the index of a target value, or return -1 if it is absent.

Example test case: [10,3,40,20,50,80,70], x=40 → 2

Hint / algorithm: At each step check mid and its immediate neighbors.

Java solution or implementation pattern:

```
static int searchNearlySorted(int[] a,int x){int l=0,r=a.length-1;while(l<=r){int m=l+(r-l)/2;if(a[m]==x)return m;if(m-1>=l&&a[m-1]==x)return
```

18. Power(x, n)

Problem statement: Implement exponentiation to calculate x raised to integer power n. Support negative exponents and handle n = -2^31 safely.

Example test case: x=2, n=-2 → 0.25

Hint / algorithm: Binary exponentiation; widen exponent to long to handle minimum int.

Java solution or implementation pattern:

```
static double myPow(double x,int n){long e=n;if(e<0){x=1/x;e=-e;}double ans=1;while(e>0){if((e&1)==1)ans*=x;x*=x;e>>=1;}return ans;}
```

19. Maximum Subarray

Problem statement: Given an integer array, find the maximum possible sum of any non-empty contiguous subarray.

Example test case: [-2,1,-3,4,-1,2,1,-5,4] → 6

Hint / algorithm: Kadane's algorithm.

Java solution or implementation pattern:

```
static int maxSubArray(int[] a){int cur=a[0],best=a[0];for(int i=1;i<a.length;i++){cur=Math.max(a[i],cur+a[i]);best=Math.max(best,cur);}return
```

20. Best Time to Buy and Sell Stock

Problem statement: Given daily stock prices, choose one day to buy and a later day to sell. Return the maximum possible profit; if no profit is possible, return 0.

Example test case: [7,1,5,3,6,4] → 5

Hint / algorithm: Track minimum price seen and best profit.

Java solution or implementation pattern:

```
static int maxProfit(int[] p){int min=Integer.MAX_VALUE,best=0;for(int x:p){min=Math.min(min,x);best=Math.max(best,x-min);}return best;}
```

21. Merge Two Sorted Arrays In-Place

Problem statement: Given two sorted arrays, merge their values so that the combined values remain sorted. Do not use a third array; rearrange the two input arrays in-place.

Example test case: a=[1,4,7], b=[2,3,6] → a=[1,2,3], b=[4,6,7]

Hint / algorithm: Gap method repeatedly compares elements across both arrays.

Java solution or implementation pattern:

```
static void merge(int[] a,int[] b){int n=a.length,m=b.length;for(int gap=nextGap(n+m);gap>0;gap=nextGap(gap)){for(int i=0,j=gap;j<n+m;i++,j++)
```

22. Find Peak Element

Problem statement: Given an array of distinct adjacent values, find the index of any peak element. A peak is strictly greater than its existing neighbors.

Example test case: [1,2,3,1] → 2

Hint / algorithm: Binary search on slope.

Java solution or implementation pattern:

```
static int findPeak(int[] a){int l=0,r=a.length-1;while(l<r){int m=l+(r-l)/2;if(a[m]>a[m+1])r=m;else l=m+1;}return l;}
```

23. Leaders in an Array

Problem statement: An element is a leader if it is greater than or equal to every element to its right. Return all leaders in their original left-to-right order.

Example test case: [16,17,4,3,5,2] → [17,5,2]

Hint / algorithm: Scan from right, collect leaders, reverse the result.

Java solution or implementation pattern:

```
static List<Integer> leaders(int[] a){List<Integer> out=new ArrayList<>();int max=Integer.MIN_VALUE;for(int i=a.length-1;i>=0;i--)if(a[i]>=max
```

24. Equilibrium Index

Problem statement: Find an equilibrium position in an array: the sum of elements strictly to its left equals the sum of elements strictly to its right. Return the 1-based position, or -1 if none exists.

Example test case: [1,3,5,2,2] → 3

Hint / algorithm: Maintain total sum and running left sum.

Java solution or implementation pattern:

```
static int equilibrium(int[] a){long total=0,left=0;for(int x:a)total+=x;for(int i=0;i<a.length;i++){total-=a[i];if(left==total)return i+1;lef
```

25. Subarray with Given Sum (Positive Values)

Problem statement: Given an array of positive integers and a target sum, find a contiguous subarray whose sum equals the target. Return its 1-based start and end positions, or -1 if no such subarray exists.

Example test case: [1,2,3,7,5], target=12 → [2,4]

Hint / algorithm: Sliding window; return 1-based endpoints or empty if absent.

Java solution or implementation pattern:

```
static int[] subarraySum(int[] a,int target){int l=0;long sum=0;for(int r=0;r<a.length;r++){sum+=a[r];while(l<=r&&sum>target)sum-=a[l++];if(su
```

MEDIUM — QUESTIONS 26–50

26. Two Sum (HashMap optimized)

Problem statement: Given an integer array and a target, return indices of two distinct elements that add up to the target. Use a hash map to achieve expected O(n) time.

Example test case: [2,7], target=9 → [0,1]

Hint / algorithm: HashMap lookup while scanning.

Java solution or implementation pattern:

```
static int[] twoSum(int[] a,int t){Map<Integer,Integer> m=new HashMap<>();for(int i=0;i<a.length;i++){if(m.containsKey(t-a[i]))return new int
```

27. Longest Subarray Sum K

Problem statement: Given an integer array that may contain positive, negative, and zero values, find the maximum length of a contiguous subarray whose sum equals k.

Example test case: [10,5,2,7,1,9], k=15 → 4

Hint / algorithm: Prefix sum → earliest index map.

Java solution or implementation pattern:

```
static int longestSumK(int[]a,int k){Map<Long,Integer> m=new HashMap<>();m.put(0L,-1);long s=0;int best=0;for(int i=0;i<a.length;i++){s+=a[i],
```

28. Subarray Sum Equals K (Count)

Problem statement: Given an integer array and an integer k, count the number of contiguous subarrays whose sum is exactly k. Negative values may appear.

Example test case: [1,1,1], k=2 → 2

Hint / algorithm: Prefix-sum frequency map.

Java solution or implementation pattern:

```
static int subarraySumCount(int[]a,int k){Map<Integer,Integer>m=new HashMap<>();m.put(0,1);int s=0,c=0;for(int x:a){s+=x;c+=m.getOrDefault(s-
```

29. Maximum Subarray (Kadane)

Problem statement: Given an integer array, calculate the maximum sum among all non-empty contiguous subarrays in O(n) time.

Example test case: [-2,1,-3,4] → 4

Hint / algorithm: Kadane's algorithm.

Java solution or implementation pattern:

```
static int maxSubArray(int[]a){int c=a[0],b=c;for(int i=1;i<a.length;i++){c=Math.max(a[i],c+a[i]);b=Math.max(b,c);}return b;}
```

30. Majority Element (Boyer-Moore)

Problem statement: Given an array with a guaranteed majority element, return the value appearing more than half the time. Aim for O(n) time and O(1) extra space.

Example test case: [3,2,3] → 3

Hint / algorithm: Boyer–Moore candidate voting.

Java solution or implementation pattern:

```
static int majorityElement(int[]a){int c=0,v=0;for(int x:a){if(c==0)v=x;c+=x==v?1:-1;}return v;}
```

31. Longest Consecutive Sequence

Problem statement: Given an unsorted integer array, return the length of its longest sequence of consecutive integer values. The consecutive values do not need to be adjacent in the input.

Example test case: [100,4,200,1,3,2] → 4

Hint / algorithm: HashSet; expand only from sequence starts.

Java solution or implementation pattern:

```
static int longestConsecutive(int[]a){Set<Integer>s=new HashSet<>();for(int x:a)s.add(x);int best=0;for(int x:s)if(!s.contains(x-1)){int y=x;v
```

32. Next Greater Element I

Problem statement: For every value in a query array, find the first greater value to its right in a reference array. If no greater value exists, use -1.

Example test case: nums1=[4,1,2], nums2=[1,3,4,2] → [-1,3,-1]

Hint / algorithm: Monotonic stack builds value-to-next-greater map.

Java solution or implementation pattern:

```
static int[] nextGreater(int[]nums1,int[]nums2){Map<Integer,Integer>m=new HashMap<>();Deque<Integer>st=new ArrayDeque<>();for(int x:nums2){whi
```

33. Next Greater Element II

Problem statement: For every index in a circular array, find the next greater value encountered when moving right, wrapping around to the beginning if needed. Use -1 if none exists.

Example test case: [1,2,1] → [2,-1,2]

Hint / algorithm: Scan indices twice using a decreasing stack of indices.

Java solution or implementation pattern:

```
static int[] nextGreaterCircular(int[]a){int n=a.length;int[]r=new int[n];Arrays.fill(r,-1);Deque<Integer>s=new ArrayDeque<>();for(int i=0;i<2
```

34. Valid Parentheses

Problem statement: Given a string containing only the characters (), [], and {}, determine whether the brackets are correctly matched and nested.

Example test case: "([])" → true

Hint / algorithm: Stack of opening brackets.

Java solution or implementation pattern:

```
static boolean validParentheses(String s){Deque<Character>st=new ArrayDeque<>();for(char c:s.toCharArray()){if(c=='('||c=='['||c=='{')st.push(
```

35. Stock Buy/Sell (Multiple Transactions)

Problem statement: Given stock prices by day, calculate the maximum profit when any number of buy/sell transactions are allowed, but you must sell before buying again.

Example test case: [7,1,5,3,6,4] → 7

Hint / algorithm: Add every positive adjacent price increase.

Java solution or implementation pattern:

```
static int maxProfitMany(int[]p){int ans=0;for(int i=1;i<p.length;i++)ans+=Math.max(0,p[i]-p[i-1]);return ans;}
```

36. Rotate Matrix 90 Degrees

Problem statement: Given an n × n matrix, rotate it 90 degrees clockwise in-place.

Example test case: [[1,2],[3,4]] → [[3,1],[4,2]]

Hint / algorithm: Transpose, then reverse each row.

Java solution or implementation pattern:

```
static void rotate(int[][]a){int n=a.length;for(int i=0;i<n;i++)for(int j=i+1;j<n;j++){int t=a[i][j];a[i][j]=a[j][i];a[j][i]=t;}for(int[]row:a
```

37. Spiral Matrix Traversal

Problem statement: Given a matrix, return its elements in clockwise spiral order, starting at the top-left corner.

Example test case: [[1,2,3],[4,5,6],[7,8,9]] → [1,2,3,6,9,8,7,4,5]

Hint / algorithm: Shrink top, bottom, left, and right boundaries.

Java solution or implementation pattern:

```
static List<Integer> spiral(int[][]a){List<Integer>r=new ArrayList<>();if(a.length==0)return r;int top=0,bottom=a.length-1,left=0,right=a[0].
```

38. Search Rotated Sorted Array

Problem statement: Given a strictly increasing array that has been rotated, search for a target and return its index, or -1 if absent. The target must be found in O(log n) time.

Example test case: [4,5,6,7,0,1,2], target=0 → 4

Hint / algorithm: At each step identify the sorted half.

Java solution or implementation pattern:

```
static int searchRotated(int[]a,int t){int l=0,r=a.length-1;while(l<=r){int m=l+(r-l)/2;if(a[m]==t)return m;if(a[l]<=a[m]){if(a[l]<=t&&t<a[m]
```

39. Merge Intervals

Problem statement: Given a list of intervals [start, end], merge all intervals that overlap and return the resulting non-overlapping intervals.

Example test case: [[1,3],[2,6],[8,10]] → [[1,6],[8,10]]

Hint / algorithm: Sort by start and merge with last output interval.

Java solution or implementation pattern:

```
static int[][] mergeIntervals(int[][]a){Arrays.sort(a,(x,y)->Integer.compare(x[0],y[0]));List<int[]>r=new ArrayList<>();for(int[]v:a){if(r.isE
```

40. Set Matrix Zeroes

Problem statement: Given a matrix, if any cell contains zero, set every cell in that cell's row and column to zero. Perform the transformation in-place.

Example test case: [[1,1,1],[1,0,1],[1,1,1]] → [[1,1,1],[1,0,1],[1,1,1]] with middle row/column zeroed

Hint / algorithm: Use first row and column as markers.

Java solution or implementation pattern:

```
static void setZeroes(int[][]a){int m=a.length,n=a[0].length;boolean fr=false,fc=false;for(int j=0;j<n;j++)if(a[0][j]==0)fr=true;for(int i=0;i
```

41. Kth Largest Element

Problem statement: Given an integer array and k, return the kth-largest element, counting duplicate values as separate positions in the sorted order.

Example test case: [3,2,1,5,6,4], k=2 → 5

Hint / algorithm: Maintain a min-heap of size k.

Java solution or implementation pattern:

```
static int kthLargest(int[]a,int k){PriorityQueue<Integer>q=new PriorityQueue<>();for(int x:a){q.offer(x);if(q.size()>k)q.poll();}return q.pee
```

42. Top K Frequent Elements

Problem statement: Given an integer array and k, return the k values that occur most frequently. Any order is acceptable unless otherwise specified.

Example test case: [1,1,1,2,2,3], k=2 → [1,2]

Hint / algorithm: Count values and use a frequency bucket.

Java solution or implementation pattern:

```
static int[] topKFrequent(int[]a,int k){Map<Integer,Integer>m=new HashMap<>();for(int x:a)m.put(x,m.getOrDefault(x,0)+1);List<Integer>[]b=new
```

43. Find Duplicate (Cycle Detection)

Problem statement: An array of length n+1 contains numbers from 1 through n with a repeated value. Find the duplicate using cycle detection without modifying the array.

Example test case: [1,3,4,2,2] → 2

Hint / algorithm: Floyd tortoise-and-hare cycle detection.

Java solution or implementation pattern:

```
static int findDuplicate(int[]a){int s=a[0],f=a[0];do{s=a[s];f=a[a[f]];}while(s!=f);s=a[0];while(s!=f){s=a[s];f=a[f];}return s;}
```

44. Trapping Rain Water

Problem statement: Given non-negative bar heights representing an elevation map, calculate how much rainwater can be trapped after raining.

Example test case: [0,1,0,2,1,0,1,3,2,1,2,1] → 6

Hint / algorithm: Two pointers with left/right maximums.

Java solution or implementation pattern:

```
static int trap(int[]h){int l=0,r=h.length-1,lm=0,rm=0,ans=0;while(l<r){if(h[l]<h[r]){lm=Math.max(lm,h[l]);ans+=lm-h[l++];}else{rm=Math.max(rm
```

45. Count Inversions

Problem statement: Count inversions in an array. An inversion is a pair of indices i < j for which a[i] > a[j].

Example test case: [2,4,1,3,5] → 3

Hint / algorithm: Merge-sort counting; see full method pattern in the code section.

Java solution or implementation pattern:

```
static long inversions(int[]a){return sortCount(a,0,a.length-1);} static long sortCount(int[]a,int l,int r){if(l>=r)return 0;int m=(l+r)/2;lon
```

46. Power(x,n) Binary Exponentiation

Problem statement: Implement x raised to n using binary exponentiation. Return a floating-point result and support negative n.

Example test case: 2^10 → 1024

Hint / algorithm: Square-and-multiply; handle negative exponent.

Java solution or implementation pattern:

```
static double pow(double x,int n){long e=n;if(e<0){x=1/x;e=-e;}double a=1;while(e>0){if((e&1)==1)a*=x;x*=x;e>>=1;}return a;}
```

47. Longest Common Prefix

Problem statement: Given an array of strings, find the longest prefix shared by every string. Return an empty string if no common prefix exists.

Example test case: ["flower","flow","flight"] → "fl"

Hint / algorithm: Shrink the first string until each string starts with it.

Java solution or implementation pattern:

```
static String longestCommonPrefix(String[]a){if(a.length==0)return "";String p=a[0];for(int i=1;i<a.length;i++)while(!a[i].startsWith(p))p=p.s
```

48. Find Peak Element

Problem statement: Given an array, return the index of any element that is greater than its adjacent elements. Either boundary has only one neighbor.

Example test case: [1,2,1] → 1

Hint / algorithm: Binary search based on adjacent slope.

Java solution or implementation pattern:

```
static int peak(int[]a){int l=0,r=a.length-1;while(l<r){int m=(l+r)/2;if(a[m]>a[m+1])r=m;else l=m+1;}return l;}
```

49. Aggressive Cows

Problem statement: Given stall positions and k cows, place the cows in stalls so the minimum distance between any two cows is as large as possible. Return that maximum minimum distance.

Example test case: stalls=[1,2,4,8,9], k=3 → 3

Hint / algorithm: Binary search distance; greedily test feasibility.

Java solution or implementation pattern:

```
static int aggressiveCows(int[]p,int k){Arrays.sort(p);int l=0,r=p[p.length-1]-p[0],ans=0;while(l<=r){int m=l+(r-l)/2;if(canPlace(p,k,m)){ans=
```

50. Minimum Platforms

Problem statement: Given train arrival and departure times for the same day, determine the minimum number of platforms required so that no train has to wait for a platform.

Example test case: arr=[900,940,950,1100,1500,1800], dep=[910,1200,1120,1130,1900,2000] → 3

Hint / algorithm: Sort arrivals and departures; sweep two pointers.

Java solution or implementation pattern:

```
static int minPlatforms(int[]a,int[]d){Arrays.sort(a);Arrays.sort(d);int i=0,j=0,cur=0,best=0;while(i<a.length){if(a[i]<=d[j]){cur++;best=Mat
```

50. 0/1 Knapsack

Problem statement: Given train arrival and departure times for the same day, determine the minimum number of platforms required so that no train has to wait for a platform.

Example test case: wt=[1,3,4], val=[15,20,30], cap=4 → 35

Hint / algorithm: 1-D DP updated backward.

Java solution or implementation pattern:

```
static int knapsack(int[] wt,int[] val,int cap){int[]d=new int[cap+1];for(int i=0;i<wt.length;i++)for(int c=cap;c>=wt[i];c--)d[c]=Math.max(d[c
```

HARD — QUESTIONS 51–75

51. Unbounded Knapsack

Problem statement: Given weights and values of n items and a knapsack capacity W, choose each item at most once to maximize total value without exceeding W.

Example test case: weights=[2,3], values=[4,5], cap=6 → 12

Hint / algorithm: Capacity loops forward for each item.

Java solution or implementation pattern:

```
static int unboundedKnapsack(int cap,int[]wt,int[]val){int[]d=new int[cap+1];for(int c=0;c<=cap;c++)for(int i=0;i<wt.length;i++)if(wt[i]<=c)d
```

52. Subset Sum

Problem statement: Given items with weights and values and a capacity W, maximize total value when each item may be chosen any number of times.

Example test case: [3,34,4,12,5,2], target=9 → true

Hint / algorithm: Boolean 1-D DP updated backward.

Java solution or implementation pattern:

```
static boolean subsetSum(int[]a,int t){boolean[]d=new boolean[t+1];d[0]=true;for(int x:a)for(int s=t;s>=x;s--)d[s]|=d[s-x];return d[t];}
```

53. Count Subsets with Given Sum

Problem statement: Given a set of non-negative integers and a target sum, determine whether some subset adds exactly to the target.

Example test case: [1,2,3,3], target=6 → 3

Hint / algorithm: Count DP, reverse sum iteration; use long if counts may be large.

Java solution or implementation pattern:

```
static long countSubsets(int[]a,int t){long[]d=new long[t+1];d[0]=1;for(int x:a)for(int s=t;s>=x;s--)d[s]+=d[s-x];return d[t];}
```

54. Partition Equal Subset Sum

Problem statement: Given a list of non-negative integers and a target sum, count the subsets whose elements add exactly to the target. Treat equal-valued elements at different indices as separate items.

Example test case: [1,5,11,5] → true

Hint / algorithm: If total is even, solve subset sum for total/2.

Java solution or implementation pattern:

```
static boolean canPartition(int[]a){int sum=0;for(int x:a)sum+=x;if(sum%2!=0)return false;boolean[]d=new boolean[sum/2+1];d[0]=true;for(int x
```

55. Coin Change Minimum Coins

Problem statement: Given a non-negative integer array, determine whether it can be divided into two subsets with equal sums.

Example test case: coins=[1,2,5], amount=11 → 3

Hint / algorithm: Unbounded minimum DP.

Java solution or implementation pattern:

```
static int coinChange(int[]c,int amount){int[]d=new int[amount+1];Arrays.fill(d,amount+1);d[0]=0;for(int s=1;s<=amount;s++)for(int x:c)if(x<=s
```

56. Coin Change Number of Ways

Problem statement: Given coin denominations and an amount, return the minimum number of coins needed to make the amount. Each coin can be used unlimited times; return -1 if impossible.

Example test case: coins=[1,2,5], amount=5 → 4

Hint / algorithm: Coins outermost to avoid permutation overcounting.

Java solution or implementation pattern:

```
static long coinWays(int[]c,int amount){long[]d=new long[amount+1];d[0]=1;for(int x:c)for(int s=x;s<=amount;s++)d[s]+=d[s-x];return d[amount]
```

57. Rod Cutting

Problem statement: Given coin denominations and an amount, count the number of combinations that make the amount when each denomination can be used unlimited times. Different orders of the same coins count as one combination.

Example test case: prices=[2,5,7,8], n=4 → 10

Hint / algorithm: Unbounded knapsack over piece lengths.

Java solution or implementation pattern:

```
static int rodCut(int[]price,int n){int[]d=new int[n+1];for(int len=1;len<=n;len++)for(int cut=1;cut<=len;cut++)d[len]=Math.max(d[len],price[
```

58. House Robber I

Problem statement: Given prices for rod pieces of lengths 1 through n and a rod of length n, determine the maximum revenue obtainable by cutting and selling the rod.

Example test case: [2,7,9,3,1] → 12

Hint / algorithm: DP with previous two states.

Java solution or implementation pattern:

```
static int rob(int[]a){int prev2=0,prev1=0;for(int x:a){int cur=Math.max(prev1,prev2+x);prev2=prev1;prev1=cur;}return prev1;}
```

59. House Robber II

Problem statement: A row of houses contains non-negative amounts of money. Rob houses to maximize the total without robbing two adjacent houses.

Example test case: [2,3,2] → 3

Hint / algorithm: Maximum of robbing ranges excluding first or excluding last.

Java solution or implementation pattern:

```
static int robCircle(int[]a){if(a.length==1)return a[0];return Math.max(robRange(a,0,a.length-2),robRange(a,1,a.length-1));} static int robRa
```

60. Longest Increasing Subsequence O(n²)

Problem statement: Houses are arranged in a circle, so the first and last houses are adjacent. Find the maximum amount that can be robbed without choosing adjacent houses.

Example test case: [10,9,2,5,3,7,101,18] → 4

Hint / algorithm: DP ending at each index.

Java solution or implementation pattern:

```
static int lis(int[]a){int n=a.length;if(n==0)return 0;int[]d=new int[n];Arrays.fill(d,1);int best=1;for(int i=0;i<n;i++)for(int j=0;j<i;j++)
```

61. Longest Increasing Subsequence O(n log n)

Problem statement: Given an integer array, return the length of its longest strictly increasing subsequence. The subsequence need not be contiguous.

Example test case: [0,1,0,3,2,3] → 4

Hint / algorithm: Maintain smallest tail for each subsequence length.

Java solution or implementation pattern:

```
static int lisFast(int[]a){int[]t=new int[a.length];int n=0;for(int x:a){int l=0,r=n;while(l<r){int m=(l+r)/2;if(t[m]<x)l=m+1;else r=m;}t[l]=
```

62. Largest Rectangle in Histogram

Problem statement: Solve the longest increasing subsequence problem in O(n log n) time.

Example test case: [2,1,5,6,2,3] → 10

Hint / algorithm: Monotonic increasing index stack with sentinel iteration.

Java solution or implementation pattern:

```
static int largestRectangle(int[]h){Deque<Integer>s=new ArrayDeque<>();int best=0;for(int i=0;i<=h.length;i++){int cur=i==h.length?0:h[i];whi
```

63. Maximal Rectangle

Problem statement: Given non-negative histogram bar heights of equal width, find the area of the largest rectangle that fits entirely under the histogram.

Example test case: [[1,0,1,0,0],[1,0,1,1,1],[1,1,1,1,1]] → 6

Hint / algorithm: Treat each row as histogram heights and apply largest-rectangle algorithm.

Java solution or implementation pattern:

```
For each row: heights[j] = cell=='1' ? heights[j]+1 : 0; update answer with largestRectangle(heights).
```

64. Sliding Window Maximum

Problem statement: Given a binary matrix, find the area of the largest rectangle containing only 1s.

Example test case: [1,3,-1,-3,5,3,6,7], k=3 → [3,3,5,5,6,7]

Hint / algorithm: Deque stores indices in decreasing value order.

Java solution or implementation pattern:

```
static int[] maxWindow(int[]a,int k){int[]r=new int[a.length-k+1];Deque<Integer>d=new ArrayDeque<>();for(int i=0;i<a.length;i++){while(!d.isEm
```

65. Allocate Minimum Pages

Problem statement: Given an array and a window size k, return the maximum value in each contiguous window of size k.

Example test case: [12,34,67,90], students=2 → 113

Hint / algorithm: Binary search answer; greedy feasibility check.

Java solution or implementation pattern:

```
static int allocatePages(int[]a,int students){if(students>a.length)return -1;int l=0,r=0;for(int x:a){l=Math.max(l,x);r+=x;}int ans=r;while(l<
```

66. Aggressive Cows (repeat)

Problem statement: Given an array where each value is the number of pages in a book and k students, allocate contiguous groups of books so every student gets at least one book and the maximum pages assigned to any student is minimized. Return -1 if allocation is impossible.

Example test case: [1,2,4,8,9], k=3 → 3

Hint / algorithm: Binary search distance + greedy placement.

Java solution or implementation pattern:

```
Use the aggressiveCows/canPlace methods from Question 49.
```

67. Word Break

Problem statement: Given stall positions and k cows, find the largest minimum distance between any two placed cows. Return the maximum distance achievable.

Example test case: s="leetcode", dict=[leet,code] → true

Hint / algorithm: DP over prefixes; try dictionary words ending at each position.

Java solution or implementation pattern:

```
static boolean wordBreak(String s,Set<String>d){boolean[]ok=new boolean[s.length()+1];ok[0]=true;for(int i=1;i<=s.length();i++)for(int j=0;j<i
```

68. Edit Distance

Problem statement: Given a string and a dictionary of words, determine whether the string can be segmented into a sequence of dictionary words.

Example test case: "horse", "ros" → 3

Hint / algorithm: Classic 2-D dynamic programming.

Java solution or implementation pattern:

```
static int editDistance(String a,String b){int[][]d=new int[a.length()+1][b.length()+1];for(int i=0;i<=a.length();i++)d[i][0]=i;for(int j=0;j<
```

69. Minimum Subset Sum Difference

Problem statement: Given two strings, find the minimum number of single-character insertions, deletions, and substitutions needed to change the first string into the second.

Example test case: [1,6,11,5] → 1

Hint / algorithm: Subset-sum reachability up to total/2.

Java solution or implementation pattern:

```
static int minSubsetDiff(int[]a){int sum=0;for(int x:a)sum+=x;boolean[]d=new boolean[sum/2+1];d[0]=true;for(int x:a)for(int s=sum/2;s>=x;s--)d
```

70. Matrix Chain Multiplication

Problem statement: Divide an array into two subsets so the absolute difference between their sums is minimized. Return that minimum difference.

Example test case: dimensions=[40,20,30,10,30] → 26000

Hint / algorithm: Interval DP over split points.

Java solution or implementation pattern:

```
static int mcm(int[]p){int n=p.length;int[][]d=new int[n][n];for(int len=2;len<n;len++)for(int i=1;i+len-1<n;i++){int j=i+len-1;d[i][j]=Intege
```

71. Count Inversions (Merge Sort)

Problem statement: Given dimensions of a chain of matrices, determine the minimum number of scalar multiplications needed to multiply the full chain. You may choose the parenthesization.

Example test case: [2,4,1,3,5] → 3

Hint / algorithm: Use merge-sort count method from Question 45.

Java solution or implementation pattern:

```
Use the inversions/sortCount methods from Question 45.
```

72. Target Sum

Problem statement: Count inversions in an array using a merge-sort-based O(n log n) algorithm.

Example test case: [1,1,1,1,1], target=3 → 5

Hint / algorithm: Transform to subset-sum count where subset sum=(sum+target)/2.

Java solution or implementation pattern:

```
static int findTargetSumWays(int[]a,int target){int sum=0;for(int x:a)sum+=x;if(Math.abs(target)>sum||(sum+target)%2!=0)return 0;int t=(sum+ta
```

73. Unique Paths

Problem statement: Given non-negative numbers and a target, assign either '+' or '-' before each number so the expression equals the target. Count the number of assignments.

Example test case: m=3,n=7 → 28

Hint / algorithm: Grid DP or 1-D row DP.

Java solution or implementation pattern:

```
static int uniquePaths(int m,int n){int[]d=new int[n];Arrays.fill(d,1);for(int i=1;i<m;i++)for(int j=1;j<n;j++)d[j]+=d[j-1];return d[n-1];}
```

74. Decode Ways

Problem statement: A grid has m rows and n columns. Starting at the top-left cell, count paths to the bottom-right cell when you can move only right or down.

Example test case: "226" → 3

Hint / algorithm: DP based on valid one- and two-digit choices.

Java solution or implementation pattern:

```
static int numDecodings(String s){if(s.isEmpty()||s.charAt(0)=='0')return 0;int prev2=1,prev1=1;for(int i=1;i<s.length();i++){int cur=0;if(s.c
```

75. House Robber III (Tree DP)

Problem statement: A digit string maps 1 to A, 2 to B, ..., 26 to Z. Count the number of ways to decode the string; a leading zero or an invalid zero pairing cannot be decoded.

Example test case: root=[3,2,3,null,3,null,1] → 7

Hint / algorithm: Return pair: take node, skip node.

Java solution or implementation pattern:

```
Use TreeNode class; recursively return {node.val+left[1]+right[1], max(left)+max(right)} for take/skip states.
```

COMPLEX — QUESTIONS 76–100

76. Binary Tree Maximum Path Sum

Problem statement: Each node of a binary tree contains money. Find the maximum sum obtainable by robbing nodes such that no node and its direct child are both selected.

Example test case: root=[-10,9,20,null,null,15,7] → 42

Hint / algorithm: DFS returns best downward path; update global best with split path.

Java solution or implementation pattern:

```
Use DFS gain: left=max(0,gain(left)); right=max(0,gain(right)); best=max(best,node.val+left+right); return node.val+max(left,right).
```

77. Diameter of Binary Tree

Problem statement: Find the maximum sum of any path in a binary tree. A path may start and end at any nodes but must follow parent-child edges.

Example test case: [1,2,3,4,5] → 3

Hint / algorithm: DFS height; update diameter with leftHeight+rightHeight.

Java solution or implementation pattern:

```
At each node compute left/right heights; update best=max(best,left+right); return 1+max(left,right).
```

78. Lowest Common Ancestor

Problem statement: Find the diameter of a binary tree: the number of edges on the longest path between any two nodes.

Example test case: BST-independent example: root=[3,5,1,6,2,0,8], p=5,q=1 → 3

Hint / algorithm: Recursive search returns current node if it is a target; combine left/right.

Java solution or implementation pattern:

```
if node null or target return node; recurse left/right; if both non-null return node; else return non-null result.
```

79. Detect Cycle in Directed Graph

Problem statement: Given a binary tree and two existing nodes p and q, return their lowest common ancestor—the deepest node that has both in its subtree.

Example test case: 0→1, 1→2, 2→0 → cycle=true

Hint / algorithm: DFS colors: unvisited, active, completed.

Java solution or implementation pattern:

```
Use adjacency list; DFS returns true on an edge to an active (gray) node; mark completed after exploring all neighbors.
```

80. Topological Sort

Problem statement: Given a directed graph, determine whether it contains a directed cycle.

Example test case: 0→1, 0→2 → valid order [0,2,1]

Hint / algorithm: Kahn's algorithm using indegrees and queue.

Java solution or implementation pattern:

```
Compute indegrees; enqueue zero-indegree vertices; pop, append, decrement neighbors; if output size != V, graph has a cycle.
```

81. Course Schedule

Problem statement: Given a directed acyclic graph (DAG), output a topological ordering of its vertices. If a cycle exists, no topological ordering is possible.

Example test case: 2 courses, prerequisites [[1,0]] → true

Hint / algorithm: Cycle detection with Kahn's algorithm.

Java solution or implementation pattern:

```
Build edges prerequisite→course; count indegrees; return processedCourses==numCourses.
```

82. Dijkstra Algorithm

Problem statement: There are numCourses courses and prerequisite pairs [a,b], meaning b must be completed before a. Determine whether all courses can be completed.

Example test case: edges 0-1:4, 0-2:1, 2-1:2 → dist from 0 [0,3,1]

Hint / algorithm: Priority queue and stale-entry skipping.

Java solution or implementation pattern:

```
Use adjacency list of (neighbor,weight); initialize dist[source]=0; relax edges from min-heap; ignore negative weights.
```

83. Bellman-Ford / Negative Cycle

Problem statement: Given a weighted graph with non-negative edge weights and a source vertex, compute the shortest distance from the source to every vertex.

Example test case: 0→1 weight 1, 1→2 weight -2, 2→1 weight -2 → negative cycle

Hint / algorithm: Relax all edges V-1 times, then do one extra pass.

Java solution or implementation pattern:

```
Initialize source 0 and others INF; relax every edge V-1 rounds; any further relaxation means negative cycle reachable.
```

84. Floyd-Warshall

Problem statement: Given a weighted directed graph that may contain negative edge weights, compute shortest paths from a source and detect a reachable negative-weight cycle.

Example test case: matrix [[0,5],[INF,0]] remains distances 0→1=5

Hint / algorithm: Try every vertex as an intermediate node.

Java solution or implementation pattern:

```
for k,i,j: if d[i][k] and d[k][j] are finite, d[i][j]=min(d[i][j],d[i][k]+d[k][j]).
```

85. Minimum Window Substring

Problem statement: Given a weighted graph represented by a distance matrix, compute shortest distances between every pair of vertices. Use a suitable infinity value for unreachable pairs.

Example test case: s="ADOBECODEBANC", t="ABC" → "BANC"

Hint / algorithm: Sliding window with character counts.

Java solution or implementation pattern:

```
Use 256-count array, expand right; when all t characters covered, shrink left and record best window.
```

86. Longest Palindromic Substring

Problem statement: Given strings s and t, find the shortest substring of s that contains every character in t, including repeated characters with their required counts. Return an empty string if no such substring exists.

Example test case: "babad" → "bab" or "aba"

Hint / algorithm: Expand around each odd/even center.

Java solution or implementation pattern:

```
For each index expand(l=i,r=i) and (i,i+1); retain longest span.
```

87. Palindrome Partitioning II

Problem statement: Given a string, return its longest palindromic substring. If multiple answers have the same length, any one may be returned.

Example test case: "aab" → 1

Hint / algorithm: Precompute palindrome table and DP minimum cuts.

Java solution or implementation pattern:

```
pal[i][j] = chars equal and (j-i<2 or pal[i+1][j-1]); dp[i] = min cuts for prefix ending at i.
```

88. Word Break Count Ways

Problem statement: Given a string, split it into substrings so every substring is a palindrome. Return the minimum number of cuts required.

Example test case: s="catsanddog", dict=[cat,cats,and,sand,dog] → 2

Hint / algorithm: dp[i] sums dp[j] for dictionary word s[j:i].

Java solution or implementation pattern:

```
dp[0]=1; for each end i and start j<i, if dp[j]>0 and dict contains s.substring(j,i), add dp[j].
```

89. DP on Trees / Binary Tree Cameras

Problem statement: Given a string and a dictionary, count the number of ways to segment the entire string into dictionary words.

Example test case: root=[0,0,null,0,0] → 1

Hint / algorithm: Postorder states: needs camera, has camera, covered.

Java solution or implementation pattern:

```
DFS returns state; if child needs camera, place camera here; if child has camera, node covered; otherwise node needs camera; add root camera i
```

90. KMP Pattern Matching

Problem statement: Given a binary tree, place the minimum number of cameras so every node is monitored. A camera monitors its own node, its parent, and its immediate children.

Example test case: text="ababcabcabababd", pattern="ababd" → index 10

Hint / algorithm: Build LPS for pattern and use fallback on mismatch.

Java solution or implementation pattern:

```
Construct prefix table; scan text, advance matched length on match, use lps[j-1] on mismatch; record start when j==pattern.length().
```

91. Z Algorithm

Problem statement: Given a text and a pattern, find all starting positions where the pattern occurs in the text using the Knuth–Morris–Pratt (KMP) algorithm.

Example test case: pattern="aba", text="abacaba" → matches at 0,4

Hint / algorithm: Maintain a [L,R] window matching the prefix.

Java solution or implementation pattern:

```
Initialize Z; reuse prior window where possible; extend character matches; for pattern search concatenate pattern+'$'+text.
```

92. Trie Insert/Search/Prefix

Problem statement: Given a string, compute the Z-array, where Z[i] is the length of the longest substring starting at i that matches a prefix of the string. Use it to locate pattern-prefix matches where appropriate.

Example test case: insert("apple"), search("apple")=true, startsWith("app")=true

Hint / algorithm: Each node has child links and an end-of-word flag.

Java solution or implementation pattern:

```
TrieNode has TrieNode[26] child and boolean end; insert creates nodes; search/prefix walk links.
```

93. Segment Tree Range Sum

Problem statement: Implement a Trie supporting insert(word), search(word), and startsWith(prefix) for lowercase English words.

Example test case: [1,3,5], sumRange(0,2)=9; update index 1 to 2 → sum=8

Hint / algorithm: Build a segment tree; update/query in O(log n).

Java solution or implementation pattern:

```
Store sums in tree[4*n]; build recursively; point update replaces a leaf and recomputes parents; query returns overlap sum.
```

94. Fenwick Tree

Problem statement: Implement a mutable integer array supporting point updates and inclusive range-sum queries. Aim for O(log n) time per update and query using a segment tree.

Example test case: values=[1,2,3,4], prefix(3)=6

Hint / algorithm: Binary Indexed Tree uses i += i&-i and i -= i&-i.

Java solution or implementation pattern:

```
For 1-based indices: add(i,v) loops while i<=n; prefix(i) loops while i>0; range(l,r)=prefix(r)-prefix(l-1).
```

95. LRU Cache

Problem statement: Implement a Fenwick Tree (Binary Indexed Tree) that supports point updates and prefix-sum queries efficiently.

Example test case: capacity=2: put(1,1), put(2,2), get(1), put(3,3) evicts key 2

Hint / algorithm: HashMap plus doubly linked list, or LinkedHashMap.

Java solution or implementation pattern:

```
Java LinkedHashMap with accessOrder=true; remove eldest entry when capacity exceeded.
```

96. LFU Cache

Problem statement: Design an LRU cache with fixed capacity. get(key) returns the value or -1; put(key,value) inserts/updates. When full, evict the least recently used key. Operations should be O(1) on average.

Example test case: capacity=2; access frequency controls eviction

Hint / algorithm: Map key→value/frequency; map frequency→ordered keys; track min frequency.

Java solution or implementation pattern:

```
On get/put, move key between frequency buckets; evict oldest key in min-frequency bucket.
```

97. N-Queens

Problem statement: Design an LFU cache with fixed capacity. Evict the least frequently used key; when frequencies tie, evict the least recently used among them. get and put should be O(1) on average.

Example test case: n=4 → 2 solutions

Hint / algorithm: Backtracking with column and diagonal sets.

Java solution or implementation pattern:

```
Try each column per row; track col, row-col, row+col; recurse and undo state.
```

98. Sudoku Solver

Problem statement: Given n, return every arrangement of n queens on an n × n chessboard such that no two queens share a row, column, or diagonal.

Example test case: A valid partially filled 9×9 board → solved board

Hint / algorithm: Backtracking with candidate checks; optimize with bitmasks.

Java solution or implementation pattern:

```
Find empty cell, try digits 1..9 valid in row/column/box, recurse; undo if unsuccessful.
```

99. Codeforces DP Practice

Problem statement: Given a partially filled 9 × 9 Sudoku board, fill the empty cells so every row, column, and 3 × 3 box contains digits 1 through 9 exactly once.

Example test case: Example depends on selected problem; see linked problem set.

Hint / algorithm: Identify state, transition, base case, and complexity; this source is a problem listing, not one fixed problem.

Java solution or implementation pattern:

```
No single solution applies: the linked page lists many problems. Choose a specific problem before coding its Java solution.
```